

L00: Introduction

Chris Carvelli

Agenda

1. Course overview
2. Tools
3. Interactive C++ applications

About Chris

- External Lecturer
- MSc ITU - Games (2019)
- Previously
 - Programmer @ CWG
 - AI Programmer @ DR Studios
 - Senior Engineer @ modl.ai
 - Stuff you don't care
- Loves coffee, trekking and making games and weird prototypes
- chca@itu.dk



About Nicholas



About you!

W1 - Welcome to the Jungle



Agenda

1. Course overview
2. Tools
3. Interactive C++ applications

Goal

Build a game from scratch

Goal

Build a game from “scratch”

But... Why?

- Lean game programming techniques
- Implement game engine features for performance
- Theory is applicable independent of platform and framework

What this course is about

We will try to develop solid basics of C/C++, gameplay systems, and more.

We will look at A LOT of gamedev related topics, most of them barely scratching the surface.

Think about this course as a sampling buffet, just a taste of many things

- provided references
- follow-up courses
- make it part of your exam project
- just as us, we love talking about this stuff!

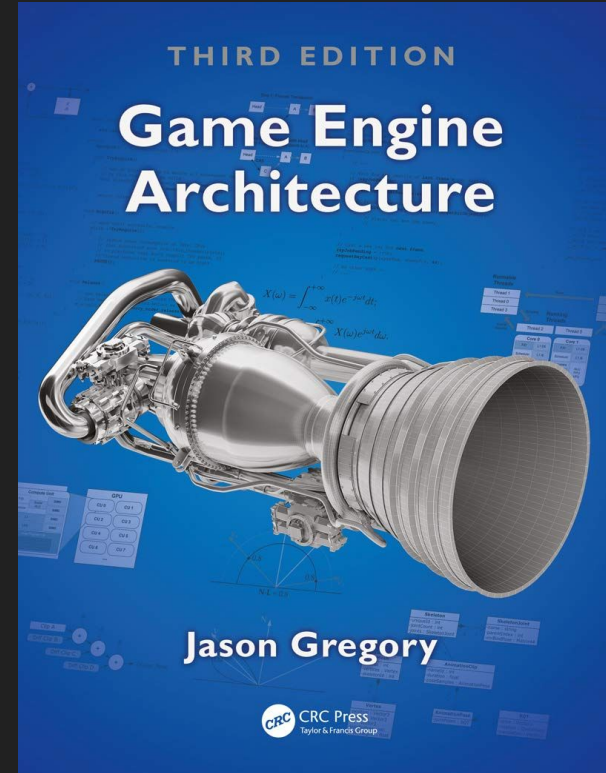
What this course is NOT about

- state of the art techniques
- cutting-edge technologies
- extreme optimization
- production-grade, shippable code code
- industry standards

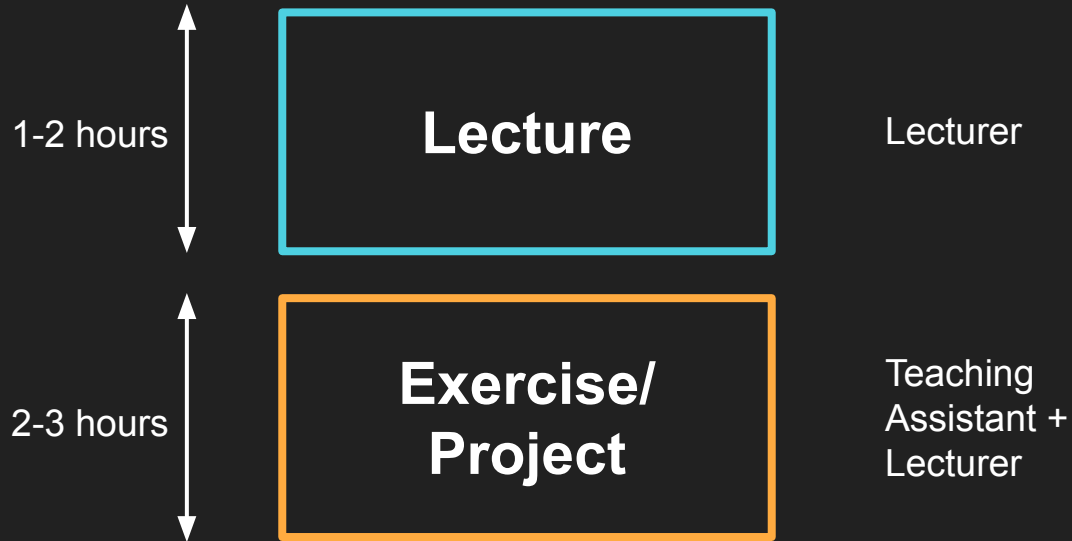
We will mention and touch some of this topics, but they won't be the focus of the course (nor part of the exam)

Course Book

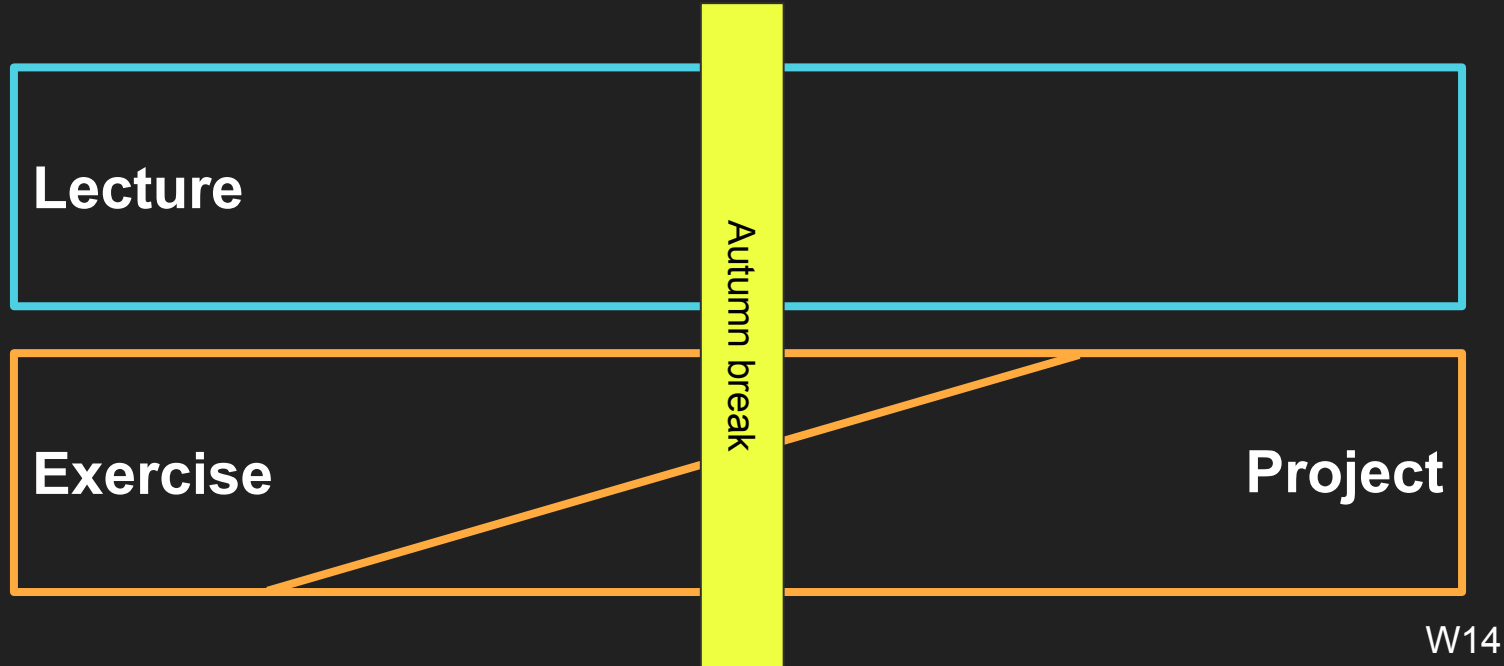
- Game Engine Architecture
- Third Edition
- Written by Jason Gregory, engine developer from Naughty Dog
- A lot of useful theory and knowledge



Class Structure



Semester Structure



No lecture on 13/10

Exercises

- Exercises every week
- Not mandatory, no hand in
- BUT you can ask us for feedback during the lab
- Split in 3 parts
 - live coding
 - main exercise
 - optional exercise
- if you can do the main exercise on your own, you are probably on the good track

Final Game Project

- Develop a small game+related project using C++
- Groups of 2-3 people
- The formal requirements will be posted on LearnIt

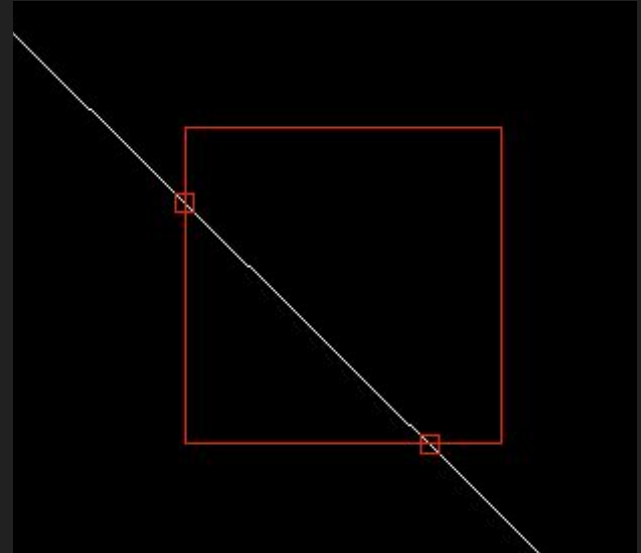
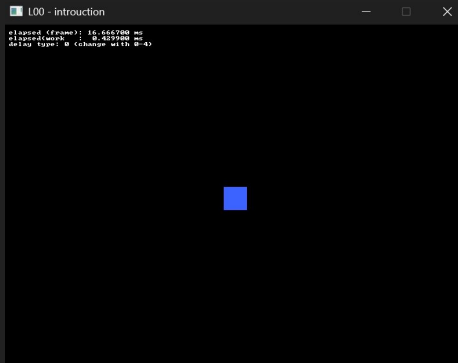
GenAI Policy

No use of generative artificial intelligence is allowed at the exam under any circumstances.

Topics (1/3)

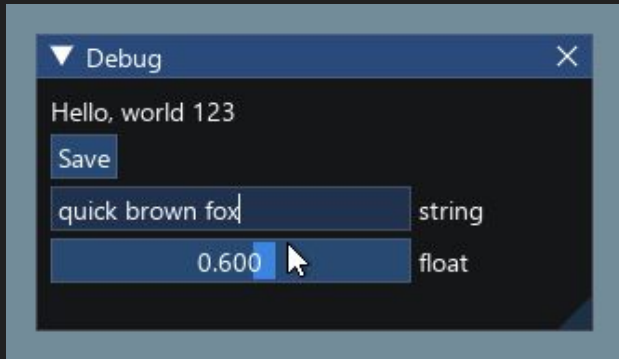
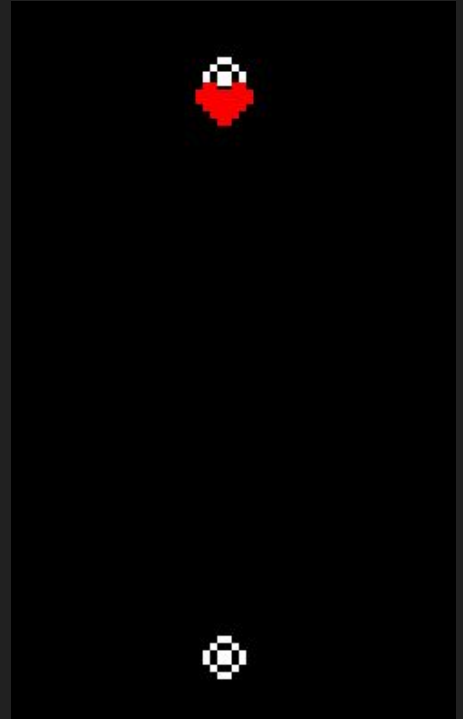
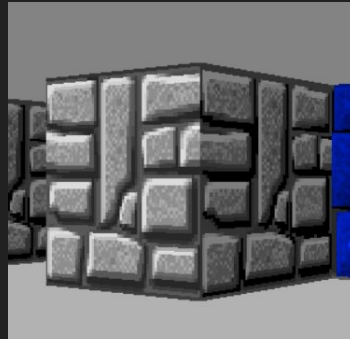


- L00: Introduction
- L01: Sprite-based Rendering
- L02: Collisions
- L03: Coordinate Systems



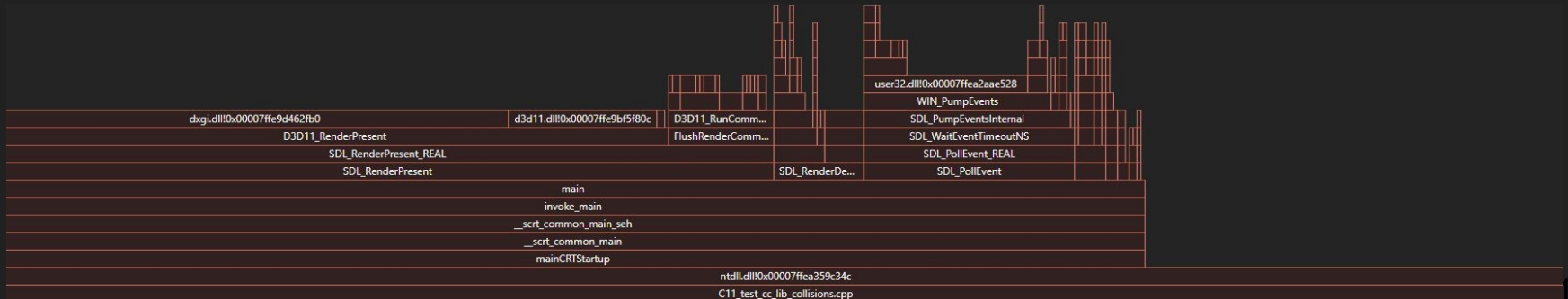
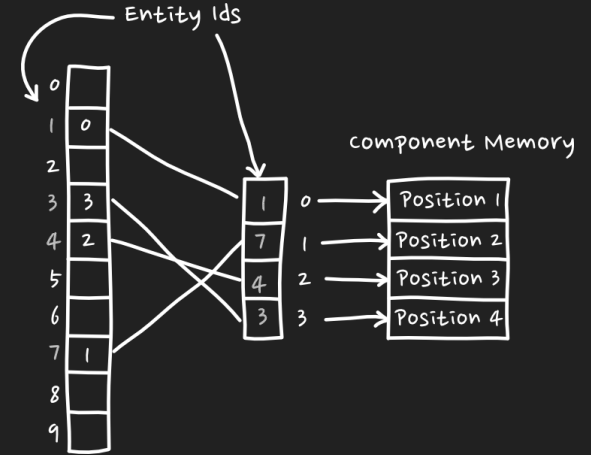
Topics (2/3)

- L04: Physics
- L05: Animation and Sound
- L06: Game UI
- L07: Scene Graphs
- L08: Client-side 3D Rendering



Topics (3/3)

- L09: Performance
- L10: Engine and Game Architecture
- L11: Advanced Topic 1 (TBD)
- L12: Advanced Topic 2 (TBD)



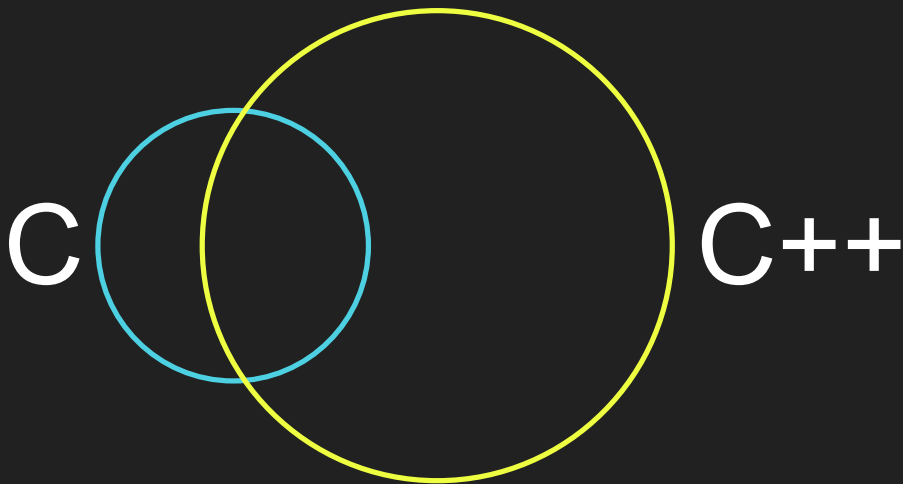
Agenda

1. Course overview
2. Tools
3. Interactive C++ applications

C vs C++

- similar (but distinct) languages
- C++ is almost a superset of C (ie, C programs will compile with a C++ compiler, unless you use few exotic features)

We will formally use C++, but we will use almost entirely C-compatible features.



<https://en.cppreference.com/index.html>

Simple DirectMedia Layer



- C-based platform-independence layer
- Pretty common in the industry
- Wraps around most of the C (*not* C++) standard library

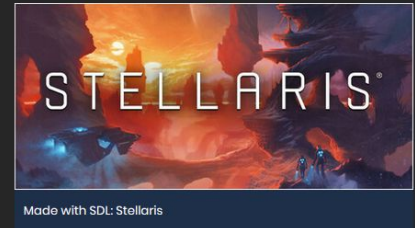
About SDL

Simple DirectMedia Layer is a cross-platform development library designed to provide low level access to audio, keyboard, mouse, joystick, and graphics hardware via OpenGL and Direct3D. It is used by video playback software, emulators, and popular games including Valve's award winning catalog and many Humble Bundle games.

SDL officially supports Windows, macOS, Linux, iOS, and Android. Support for other platforms may be found in the source code.

SDL is written in C, works natively with C++, and there are bindings available for several other languages, including C# and Python.

SDL is distributed under the [zlib license](#). This license allows you to use SDL freely in any software.



<https://www.libsdl.org/index.php>

Tools used in the course

- Version control  git
 - + free and open source
 - + very common in the wild
 - not great with binary files
 - not friendly towards non-technical users
- Build system  cmake
 - + free and open source
 - + common for open source projects in the wild
 - not great to work with
- IDE  xcode
 - + free to use (but proprietary)
 - + properly cross platform
 - + pretty good basic features (but lacks advanced debug features and any profiling)

Alternative IDEs

Visual studio

- + option of free to use licence (but not open source)
- + very good debugging and ok profiling
- + go to option for Windows development (and thus, for gamedev)
- windows only

Xcode

- + free to use (but not open source)
- + very good debugging and ok profiling (GPU graphics included)
- macOS only
- looks like it's not possible to create C++ projects anymore?
- no cmake integration

CLion

- ~~expensive, service-style paid licence only (you can get one through ITU)~~
Free for non-commercial use?
- + very good debugging and profiling
- + properly cross platform

A note on compilers

-  gcc (linux and macOS)
-  cl (windows)
-  clang (cross-platform)

For our needs, they are basically the same thing.

Live-coding sessions will be done using clang, so if you want to have an easier time following (same error messages, same warning profiles) you may want to install and use it.

Agenda

1. Course overview
2. Tools
3. Interactive C++ applications

What is a Game?

- Rules
- Challenges
- Puzzles
- Game world / Magic circle
- Video game
- Interactive experience
- World simulation (physics, economy, social behaviour)

What is a Game?

- Rules
- Challenges
- Puzzles
- Game world / Magic circle
- Video game
- Interactive experience
- World simulation (physics, economy, social behaviour)

Time and Loops

The core of any game engine is a system of multiple loops to make it so every part of the engine gets executed at the correct time and it's notified of any relevant interaction with external entities (the player, the OS, other clients in multiplayer environments, ...)

The Main Loop

- `init()`: handles initialization
- `process_events()`: handles user input, OS interrupts, ...
- `update()`: executes one simulation step for all systems and game elements
- `render()`: prepares data to send to the renderer/GPU

```
// Simple loop
int main()
{
    init();

    while(!quit) // game loop
    {
        process_events();
        update();
        render();
    }

    return 0;
}
```

Main Loop Types - Simple

- Effective, but a bit naive
- Tries to execute everything as soon as possible
- Game will go faster or slower, depending on how much computation needs to be done

```
// Simple loop
int main()
{
    init();

    while(!quit) // game loop
    {
        process_events();
        update();
        render();
    }

    return 0;
}
```

Main Loop Types - Fixed

- Partially addresses the variable computation length problem
- Needs to specify a target “speed”, expressed in milliseconds per frame
- After a single step is done, waits until the target time is elapsed.
- Makes it so the game cannot go too fast, but it could still be too slow!

```
// Fixed loop
int main()
{
    init();

    while(!quit) // game loop
    {
        float start = get_current_time();

        process_events();
        update();
        render();

        float end = get_current_time();
        float elapsed = end - start;

        sleep(MS_PER_FRAME - elapsed);
    }

    return 0;
}
```

Main Loop Types - Fluid

- Goes back running as fast as possible, but notifies the subsystems of how much time passed since last computation.
- Works very well in most cases, BUT:
 - Slightly more complex
 - Subsystems and game code needs to be written to take advantage of this
 - Extreme variations in FPS (especially when they are too low) will still create problems when dealing with user input, and in general won't look very good
 - extremely power-hungry, which not irrelevant in this age of mobile gaming (and, you know climate change...)

```
// Fluid loop
int main()
{
    init();
    float start = get_current_time();
    float delta = 0;

    while(!quit) // game loop
    {
        process_events();
        update(delta);
        render();

        float end = get_current_time();
        delta = end - start;
        start = end;
    }

    return 0;
}
```

Game Loops

```
// Simple loop
int main()
{
    init();

    while(!quit) // game loop
    {
        process_events();
        update();
        render();
    }

    return 0;
}
```

```
// Fixed loop
int main()
{
    init();

    while(!quit) // game loop
    {
        float start = get_current_time();

        process_events();
        update();
        render();

        float end = get_current_time();
        float elapsed = end - start;

        sleep(MS_PER_FRAME - elapsed);
    }

    return 0;
}
```

```
// Fluid loop
int main()
{
    init();
    float start = get_current_time();
    float delta = 0;

    while(!quit) // game loop
    {
        process_events();
        update(delta);
        render();

        float end = get_current_time();
        delta = end - start;
        start = end;
    }

    return 0;
}
```

Today's Exercise

- Get familiar with the tools used in the course
- get familiar with C/SDL
- Enjoy what's left of summer

